

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea Magistrale in
Informatica

PoS tagging italiano con BERT

Progetto di Natural Language Processing

Studente: Pierre Errico

Matricola: 0001235580

Anno Accademico 2026/2027

Abstract

Il lavoro affronta il tagging morfosintattico dell'italiano con BERT preaddestrato, assumendo la predizione delle categorie UPOS come obiettivo principale e quella delle caratteristiche FEATS come estensione. Si confrontano tre architetture: encoder congelato con classificazione lineare, fine-tuning completo con classificazione lineare e fine-tuning con CRF sulla sequenza UPOS.

Ciascuna architettura viene addestrata prima su UPOS e poi sul compito congiunto UPOS e FEATS, a partire dal checkpoint della prima fase, ottenendo sei configurazioni. La valutazione utilizza UD Italian-ISDT 2.15 con segmentazione gold e riporta accuracy e macro-F1. UDPipe 2 costituisce il riferimento esterno per il confronto UPOS.

Indice

1 Introduzione **1**
1.1 Il compito di PoS tagging 1
1.2 Le principali sfide del PoS tagging 2
1.3 Modelli neurali per l'elaborazione del linguaggio 2
1.4 Architettura BERT 3
1.4.1 Tokenizzazione 3
1.4.2 Struttura dell'encoder 3
1.4.3 Pre-addestramento 4
1.5 Obiettivi del progetto 4

2 Metodologia **6**
2.1 Disegno sperimentale 6
2.2 Dati e annotazioni 7
2.2.1 Corpus e suddivisione 7
2.2.2 Rappresentazione di UPOS e FEATS 7
2.3 Preprocessing e allineamento 8
2.4 Modelli e funzione obiettivo 8
2.5 Procedura di addestramento 9
2.6 Selezione del checkpoint 9
2.7 Valutazione 10
2.7.1 Metriche UPOS 10
2.7.2 Metriche FEATS 10
2.7.3 Aggregazione e analisi degli errori 11
2.8 Confronto con UDPipe 2 11
2.9 Riproducibilità 11

Bibliografia **12**

Capitolo 1

Introduzione

1.1 Il compito di PoS tagging

Il PoS tagging è un problema di etichettatura di sequenze di token, nel quale a ogni token viene assegnata un'etichetta che ne identifica la categoria grammaticale nel contesto frasale. Un token corrisponde generalmente a una parola; può tuttavia indicare anche altri elementi della frase, come simboli e punteggiatura. Il PoS tagging è un compito fondamentale per l'analisi linguistica e costituisce una fase preliminare per numerose applicazioni di Natural Language Processing (NLP), tra cui la lemmatizzazione, l'analisi sintattica e la traduzione automatica [1].

Nel sistema Universal Dependencies (UD), l'inventario UPOS comprende le 17 categorie riportate nella Tabella 1.1 [2]. Le caratteristiche morfologiche, come genere, numero e tempo verbale, sono rappresentate separatamente, quando applicabili, mediante gli attributi del campo `FEATS` [3].

Tag UPOS	**Categoria grammaticale**	**Esempio contestuale**
`ADJ`	Aggettivo	Il gatto è **nero**.
`ADP`	Adposizione	Vado **a** casa.
`ADV`	Avverbio	Il gatto dorme **tranquillamente**.
`AUX`	Verbo ausiliare	Maria **ha** mangiato.
`CCONJ`	Congiunzione coordinante	Leggo **e** scrivo.
`DET`	Determinante	**Il** gatto dorme.
`INTJ`	Interiezione	**Oh**, sei arrivato!
`NOUN`	Nome comune	Il **gatto** dorme.
`NUM`	Numerale	Ho **tre** libri.
`PART`	Particella	John**'s** book.
`PRON`	Pronome	**Lei** legge un libro.
`PROPN`	Nome proprio	**Maria** legge un libro.
`PUNCT`	Punteggiatura	Sei a casa**?**
`SCONJ`	Congiunzione subordinante	Resto a casa **perché** piove.
`SYM`	Simbolo	Il risultato è 2 + 3.
`VERB`	Verbo	Il gatto **dorme**.
`X`	Altro	Ha pronunciato **xfgh**.

Tabella 1.1: Categorie UPOS ed esempi contestuali. Per `PART` è illustrato il marcatore possessivo inglese; per `X`, una sequenza non interpretabile.

La frase «*Il gatto dorme.*» può essere rappresentata dalla sequenza `DET NOUN VERB PUNCT`, alla quale si affiancano, quando applicabili, le annotazioni morfologiche del campo `FEATS`; e.g., al token «*gatto*», etichettato come `NOUN`, è associato `Gender=Masc|Number=Sing` [3].

1.2 Le principali sfide del PoS tagging

Una delle principali difficoltà del PoS tagging è rappresentata dall'ambiguità grammaticale: una stessa parola può appartenere a più categorie, e la scelta del tag corretto dipende dal contesto [1]. Ad esempio, la parola *porta* può essere un nome (e.g. «*La porta è chiusa.*») o un verbo (e.g. «*Lui porta il libro.*»).

Un altro problema risiede nello sbilanciamento della distribuzione dei tag: alcune categorie, come nomi comuni (`NOUN`), determinanti (`DET`) e adposizioni (`ADP`), sono molto frequenti, mentre altre, come simboli (`SYM`) ed elementi non classificabili nelle altre categorie (`X`), compaiono raramente [4]. La presenza di classi con pochi esempi può rendere più difficile l'apprendimento di modelli robusti e generalizzabili [5].

A ciò si aggiungono la presenza di parole rare o mai osservate durante l'addestramento, come neologismi, termini specialistici e nomi propri, che il modello può quindi faticare a classificare [6]; le differenze tra il dominio lessicale dei testi di addestramento e quello dei testi da analizzare, che possono ridurre la capacità di generalizzazione del modello [1]; e gli eventuali errori o incoerenze nelle annotazioni, che possono influenzare sia l'apprendimento sia la valutazione [1].

Queste problematiche evidenziano l'importanza di adottare modelli capaci di costruire rappresentazioni linguistiche sensibili al contesto e di sfruttare conoscenze apprese da grandi quantità di dati testuali [7].

1.3 Modelli neurali per l'elaborazione del linguaggio

Naturalmente, il PoS tagging richiede modelli capaci di identificare le categorie grammaticali delle parole tenendo conto del contesto frasale. L'evoluzione delle tecniche di NLP ha progressivamente migliorato questa capacità, passando da rappresentazioni lessicali statiche ad architetture in grado di integrare informazioni contestuali sempre più complesse [8].

Tra gli approcci neurali più influenti, Word2Vec [9] ha permesso di apprendere rappresentazioni vettoriali dense delle parole, dette *word embeddings*, nelle quali termini utilizzati in contesti simili tendono ad assumere rappresentazioni vicine. Tali embedding sono tuttavia statici, i.e. associano a ciascuna parola un unico vettore indipendentemente dal contesto, rappresentando quindi un limite per il PoS tagging.

Le reti neurali ricorrenti (RNN), comprese le Long Short-Term Memory (LSTM), producono rappresentazioni dipendenti dal contesto attraverso l'elaborazione sequenziale dei token, mentre le varianti bidirezionali, come le BiLSTM, integrano informazioni provenienti da entrambe le direzioni della sequenza [8, 6]. La natura

ricorrente di questi modelli limita tuttavia la parallelizzazione e complica la gestione delle dipendenze a lungo raggio [10].

L'architettura Transformer [10] supera alcuni di questi limiti sostituendo la ricorrenza con meccanismi di attenzione. In particolare, la *Self-Attention* permette di modellare direttamente le relazioni tra i token, anche distanti tra loro, favorendo un'elaborazione parallela della sequenza e la produzione di rappresentazioni contestuali.

1.4 Architettura BERT

Tra i modelli basati sull'architettura Transformer, Bidirectional Encoder Representations from Transformers (BERT) [7] sfrutta un Transformer Encoder bidirezionale per produrre rappresentazioni contestuali dei token, risultando particolarmente adatto a compiti come il PoS tagging, nei quali la classificazione grammaticale dipende dal contesto frasale.

1.4.1 Tokenizzazione

L'input dell'encoder di BERT è costituito da token prodotti da WordPiece, un metodo di tokenizzazione che mantiene le parole intere oppure le suddivide in frammenti più piccoli, detti *subword*, selezionati da un vocabolario predefinito. Ad esempio, il tokenizer associato al modello `dbmdz/bert-base-italian-cased` rappresenta *bicicletta* con un singolo token, mentre suddivide *casetta* in `case` e `##tta`; il prefisso `##` indica la continuazione della parola. Questo approccio permette di rappresentare parole rare combinando frammenti già disponibili [7].

1.4.2 Struttura dell'encoder

Il modello `dbmdz/bert-base-italian-cased` adotta la configurazione BERT-Base, composta da 12 blocchi Transformer Encoder, 12 teste di attenzione per blocco e rappresentazioni di dimensione 768 [11]. Prima dell'elaborazione, gli embedding dei token vengono sommati a quelli posizionali e di segmento, che identificano rispettivamente la posizione nella sequenza e l'appartenenza ai segmenti di testo. Le rappresentazioni risultanti attraversano i 12 blocchi in successione, mantenendo invariata la loro dimensione [7].

Ogni blocco comprende una Multi-Head Self-Attention e una rete feed-forward. La prima esegue più operazioni di attenzione in parallelo, utilizzando proiezioni query, key e value apprese separatamente per ciascuna testa. Le uscite vengono concatenate e proiettate nuovamente nello spazio di dimensione 768 [10]. Una delle particolarità dell'architettura è l'attenzione bidirezionale: ciascun token può utilizzare sia il contesto precedente sia quello successivo, senza vincoli causali [7].

Per ciascuna testa, l'attenzione è calcolata come:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1.1}$$

dove Q, K e V sono proiezioni lineari apprese delle rappresentazioni in ingresso:

- **Query (Q):** rappresentano le informazioni ricercate da ciascun token nel contesto;
- **Key (K):** rappresentano le caratteristiche dei token utilizzate per calcolare la compatibilità con le query;
- **Value (V):** rappresentano le informazioni da aggregare mediante i pesi di attenzione.

Il termine d_k indica la dimensionalità dei vettori key, pari a $768/12 = 64$ per ciascuna testa.

La rete feed-forward applica indipendentemente a ciascun token due trasformazioni lineari, da 768 a 3 072 componenti e viceversa, separate da un'attivazione GELU. Mentre l'attenzione integra informazioni tra posizioni, questa rete elabora le rappresentazioni individualmente. Entrambi i sottolivelli impiegano dropout per ridurre l'overfitting, oltre a connessioni residue e Layer Normalization per favorire la stabilità dell'addestramento [7, 10].

Al termine dei 12 blocchi, l'encoder restituisce un vettore contestuale di 768 componenti per ciascun token, utilizzabile dalle teste di classificazione dei compiti supervisionati.

1.4.3 Pre-addestramento

Nel BERT originale il preaddestramento combina il *Masked Language Modeling* (MLM) e la *Next Sentence Prediction* (NSP). Nel MLM viene selezionato casualmente il 15% dei token della sequenza: di questi, l'80% viene sostituito con il token speciale `[MASK]`, il 10% con un token casuale e il restante 10% rimane invariato. Il modello deve predire i token originali nelle posizioni selezionate, sfruttando il contesto bidirezionale. La NSP richiede invece di determinare se due segmenti di testo siano consecutivi nel corpus, mediante una classificazione binaria [7].

Attraverso il preaddestramento, BERT apprende rappresentazioni linguistiche contestuali da grandi quantità di testo, che possono successivamente essere adattate a diversi compiti supervisionati mediante fine-tuning.

Il modello `dbmdz/bert-base-italian-cased`, basato sull'architettura BERT-Base descritta in precedenza, è preaddestrato su corpora italiani e distingue tra lettere maiuscole e minuscole [11].

1.5 Obiettivi del progetto

Il presente lavoro studia l'adattamento di BERT al PoS tagging della lingua italiana, estendendo l'analisi anche alla predizione delle caratteristiche morfologiche `FEATS`. Si

confrontano tre architetture: BERT congelato con teste di classificazione addestrabili, BERT sottoposto a fine-tuning completo e BERT fine-tuned con un Conditional Random Field (CRF) sulla sola sequenza UPOS. Le tre architetture vengono valutate prima su UPOS e poi sul compito congiunto UPOS e FEATS, ottenendo sei configurazioni. L'obiettivo principale è valutare il contributo del fine-tuning dell'encoder e della decodifica strutturata alla predizione dei tag UPOS. La predizione delle caratteristiche `FEATS` costituisce un obiettivo aggiuntivo.

Gli esperimenti utilizzano il treebank UD Italian-ISDT 2.15 e assumono parole già segmentate. UDPipe 2 costituisce il riferimento esterno per il confronto UPOS [4, 12]. Il protocollo sperimentale e le metriche di valutazione sono descritti nel capitolo dedicato alla metodologia (Capitolo 2).

Capitolo 2

Metodologia

2.1 Disegno sperimentale

Il disegno sperimentale confronta le tre architetture della Tabella 2.1, distinguendo l'effetto dell'adattamento dell'encoder da quello della decodifica strutturata UPOS. Ogni architettura viene valutata dopo una prima fase dedicata al solo compito UPOS e dopo una successiva estensione al compito congiunto UPOS e FEATS, ottenendo sei configurazioni che consentono di osservare anche la variazione delle prestazioni del compito principale durante la prosecuzione del training.

Architettura	Encoder	UPOS	FEATS (fase 2)	Learning rate
BERT congelato	Pesi fissi	Lineare	Lineari	10^{-3}
BERT fine-tuned	Addestrabile	Lineare	Lineari	$2 \cdot 10^{-5}$
BERT + CRF	Addestrabile	Lineare e CRF	Lineari	$2 \cdot 10^{-5}$

Tabella 2.1: Architetture e learning rate. Le teste presenti e l'eventuale CRF sono sempre addestrabili; le teste FEATS vengono aggiunte nella seconda fase.

Nella prima fase, a partire dal checkpoint BERT preaddestrato, l'ottimizzazione coinvolge la testa UPOS, l'eventuale CRF e, nelle architetture fine-tuned (i.e. con encoder addestrabile), l'encoder. Nella seconda fase, il checkpoint UPOS selezionato per la stessa architettura e lo stesso seed viene esteso aggiungendo le teste FEATS, inizializzate ex novo, mentre encoder, testa UPOS ed eventuale CRF mantengono i pesi appresi. L'addestramento prosegue quindi su entrambi i compiti, con il CRF applicato alla sola sequenza UPOS e le FEATS predette tramite teste lineari; nelle architetture fine-tuned, le rispettive loss contribuiscono all'aggiornamento dell'encoder condiviso [13].

Ciascuna configurazione viene eseguita con i seed 42, 123 e 456, per un totale di 18 prove, nove per fase, completando tutte le prove UPOS prima di avviare l'estensione a FEATS. L'impiego degli stessi seed permette di confrontare le configurazioni attraverso esecuzioni appaiate e di valutare la variabilità dei risultati mediante le statistiche descritte nella sezione 2.7 [14].

2.2 Dati e annotazioni

2.2.1 Corpus e suddivisione

Si utilizza UD Italian-ISDT, release `r2.15`, scaricato dal repository ufficiale [4] e distribuito in file CoNLL-U (i.e. file testuali tabulari che riportano le parole delle frasi e le relative annotazioni linguistiche) [3]. Gli split ufficiali della Tabella 2.2 sono mantenuti, utilizzando il training per aggiornare i parametri e costruire gli inventari delle classi, il development set per selezionare il checkpoint e il test per la valutazione finale su dati esclusi dalle due operazioni precedenti.

Split	Frasi	Parole	Tag UPOS presenti
Train	13 121	276 014	17
Dev	564	11 907	17
Test	482	10 417	15

Tabella 2.2: Dimensioni degli split di UD Italian-ISDT 2.15; le parole corrispondono alle righe CoNLL-U con ID intero.

La lettura del corpus conserva le parole sintattiche identificate da un ID intero, escludendo le righe relative a token multiparola e nodi vuoti [3], mentre le forme delle parole e i confini di frase vengono acquisiti direttamente dalle annotazioni. Il protocollo assume pertanto segmentazione gold e confronta tutte le configurazioni sulle medesime parole, isolando la valutazione dell'etichettatura dalle operazioni di segmentazione. Un controllo delle sequenze di forme ha inoltre rilevato una frase identica tra training e sviluppo e nessuna tra training e test, mantenendo comunque la suddivisione ufficiale.

2.2.2 Rappresentazione di UPOS e FEATS

Gli inventari delle classi sono costruiti esclusivamente sul training e comprendono 17 tag UPOS e 14 attributi FEATS, per un totale di 40 coppie attributo-valore. Ciascuna parola viene rappresentata mediante un'etichetta UPOS e un vettore di etichette morfologiche, nel quale ogni componente corrisponde a un attributo e distingue i suoi valori dalla classe `<NONE>` (i.e. assenza dell'attributo).

Per esempio, una parola annotata con `Gender=Masc|Number=Sing` riceve `Masc` nella testa Gender, `Sing` nella testa Number e `<NONE>` nelle componenti degli attributi assenti, consentendo al modello di apprendere sia la presenza degli attributi sia i rispettivi valori. Gli eventuali valori multipli di uno stesso attributo sono ordinati e conservati come un'unica etichetta composta, in accordo con il formato CoNLL-U [3].

Le metriche utilizzano le annotazioni originali anche quando contengono coppie assenti dal training (e.g. `Typo=Yes` nel test), così che questi casi rimangano nel conteggio degli errori anche quando il modello non dispone della classe corrispondente.

2.3 Preprocessing e allineamento

Il tokenizer fast di `dbmdz/bert-base-italian-cased` viene applicato alle parole presegmentate, mantenendo la corrispondenza tra ciascuna subword e la parola di origine. Il vettore delle etichette viene assegnato soltanto alla prima subword, escludendo token speciali, frammenti successivi e padding dalla loss e dalle metriche [15]; questa scelta mantiene una sola decisione supervisionata per parola, indipendentemente dal numero di frammenti prodotti dal tokenizer.

Nella variante CRF, gli score delle prime subword vengono raccolti in sequenze continue di parole e accompagnati da una maschera che identifica le posizioni valide, in modo che le transizioni siano apprese e decodificate tra parole consecutive. Il padding è dinamico per batch, mentre il limite di 512 token consente di elaborare interamente tutte le frasi del corpus; il preprocessing verifica che ogni parola sia coperta e interrompe l'esecuzione se tale condizione non è soddisfatta.

2.4 Modelli e funzione obiettivo

L'encoder produce le rappresentazioni utilizzate da una testa UPOS nella prima fase e da 15 teste nella seconda, una per UPOS e 14 per FEATS, applicando a ciascuna rappresentazione un dropout con probabilità 0.1 prima della trasformazione lineare. Le teste lineari vengono addestrate mediante cross-entropy mediata sulle parole valide e predicono la classe con score massimo, condividendo l'encoder e mantenendo parametri di classificazione distinti [15].

Nella variante con CRF, gli score della testa UPOS costituiscono le emissioni di una catena lineare, alla quale si aggiungono parametri che rappresentano le transizioni tra tag consecutivi, così da modellare regolarità locali delle sequenze grammaticali. L'addestramento minimizza la log-verosimiglianza negativa della sequenza gold, normalizzata sul numero di parole del batch, mentre la decodifica Viterbi seleziona la sequenza con score complessivo massimo [16]. La decodifica strutturata è applicata a UPOS per valutarne il contributo al compito principale, mentre le teste FEATS lineari consentono di aggiungere la supervisione morfologica mantenendo contenuta la complessità del modello.

Indicando con $\mathcal{A}$ l'insieme degli attributi morfologici, le funzioni obiettivo delle due fasi sono

$$\mathcal{L}^{(1)} = \mathcal{L}_{\text{UPOS}}, \qquad \mathcal{L}^{(2)} = \mathcal{L}_{\text{UPOS}} + \frac{\lambda}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathcal{L}_a, \qquad \lambda = 1. \tag{2.1}$$

La media delle loss FEATS mantiene il peso complessivo del compito morfologico indipendente dal numero delle teste, mentre la presenza della loss UPOS nella seconda fase consente di proseguire l'apprendimento del compito principale durante l'introduzione delle nuove supervisioni. Il coefficiente λ rimane fisso per tutte le architetture e tutti i seed, includendo nell'apprendimento anche le decisioni di assenza rappresentate da `<NONE>`.

2.5 Procedura di addestramento

L'ottimizzazione utilizza AdamW [17] attraverso il `Trainer` di Transformers [15], con i learning rate della Tabella 2.1 e le impostazioni comuni della Tabella 2.3. Il valore più elevato della variante congelata riguarda soltanto le teste, mentre nelle architetture fine-tuned il learning rate viene applicato anche all'encoder; il seed viene impostato prima della creazione del modello e controlla l'inizializzazione delle nuove teste, l'ordine dei batch e le operazioni stocastiche.

Parametro	Valore
Epoche per fase	5
Batch di training / valutazione	8 / 16 frasi
Weight decay	0.01
Warmup	10% dei passi
Clipping della norma dei gradienti	1.0

Tabella 2.3: Impostazioni comuni alle due fasi; dopo il warmup, il learning rate decresce linearmente.

Nella seconda fase si caricano i pesi del checkpoint UPOS selezionato e si avviano una nuova ottimizzazione AdamW e un nuovo scheduler per altre cinque epoche, mantenendo fisso l'encoder del baseline congelato. Le variazioni UPOS vanno quindi interpretate considerando l'effetto congiunto dell'estensione multitask e della prosecuzione dell'addestramento.

L'esecuzione su NVIDIA GeForce RTX 4070 SUPER, con 12 GB di memoria, utilizza precisione mista FP16, registrando nei metadati le impostazioni effettive dell'ambiente di calcolo.

2.6 Selezione del checkpoint

Al termine di ogni epoca il modello viene valutato sul development set e viene salvato un checkpoint, selezionando in entrambe le fasi quello che massimizza la macro-F1 UPOS sui 17 tag del training. Questo criterio mantiene la priorità del compito principale e assegna uguale peso alle categorie grammaticali, mentre le metriche FEATS vengono monitorate durante la seconda fase per descrivere l'apprendimento del compito aggiuntivo.

Il checkpoint selezionato nella prima fase costituisce anche il punto di partenza della corrispondente prova UPOS e FEATS, collegando ogni modello multitask alla propria esecuzione UPOS con lo stesso seed. La valutazione sul test viene effettuata dopo il completamento delle 18 prove e delle rispettive selezioni sul development set, riservando tali dati al confronto finale.

2.7 Valutazione

Le predizioni vengono ricondotte alle parole originali mediante l'allineamento delle prime subword, verificando la corrispondenza dell'ordine e delle etichette di riferimento; le metriche UPOS riguardano tutti e sei i modelli, mentre quelle FEATS e l'accuracy congiunta riguardano i tre modelli della seconda fase.

2.7.1 Metriche UPOS

L'accuracy misura la frazione di parole con tag corretto e descrive la prestazione complessiva, alla quale viene affiancata la macro-F1 per attribuire lo stesso peso alle categorie, comprese quelle meno frequenti. Per ciascun tag k si conteggiano veri positivi (TP_k), falsi positivi (FP_k) e falsi negativi (FN_k), dai quali si ricava l'F1 come media armonica di precision e recall [18]:

$$F_{1,k} = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad F_1^{\text{macro}} = \frac{1}{|C|} \sum_{k \in C} F_{1,k}. \tag{2.2}$$

La versione *totale* include i 17 tag del training, mentre quella sulle classi *presenti* considera soltanto i tag con supporto gold positivo nello split valutato, che nel test sono 15 per l'assenza di `INTJ` e `PART`. Ponendo a zero gli F1 con denominatore nullo, la macro-F1 totale sul test ha quindi un massimo teorico di $15/17 \simeq 88{,}24\%$ anche con tutte le predizioni corrette, mentre quella sulle classi presenti mantiene come massimo il 100%.

Considerando tutti i tag, precision, recall e F1 con media *micro* coincidono con l'accuracy, poiché ogni parola riceve una sola etichetta [18]; il riepilogo utilizza pertanto accuracy e macro-F1, che sintetizzano correttezza complessiva e prestazioni per categoria. Precision e recall delle singole classi restano disponibili per l'analisi degli errori, distinguendo la correttezza delle assegnazioni dalla copertura delle occorrenze gold.

2.7.2 Metriche FEATS

L'accuracy FEATS principale richiede che l'intero insieme di coppie attributo-valore di una parola coincida con quello gold, valutando congiuntamente la presenza degli attributi e la correttezza dei loro valori. Indicando con G_i e P_i gli insiemi gold e predetto per la parola i, si calcola

$$\text{Acc}_{\text{FEATS}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[G_i = P_i], \tag{2.3}$$

includendo anche gli insiemi vuoti, mentre una misura aggiuntiva conta le decisioni corrette sulle coppie parola-attributo e comprende la classe `<NONE>`. Quest'ultima misura viene interpretata insieme all'accuracy dell'insieme completo, poiché la frequenza degli attributi assenti può incidere sensibilmente sul suo valore; gli attributi valutati sono l'unione di quelli di training, gold e predetti.

La macro-F1 tratta ogni coppia attributo-valore (e.g. `Gender=Fem`) come una classe e ne conteggia veri positivi, falsi positivi e falsi negativi sulla presenza nelle singole parole, escludendo `<NONE>` dalla media. La versione *totale* comprende le 40 coppie del training e le ulteriori coppie osservate nelle annotazioni gold o nelle predizioni, mentre quella sulle classi *presenti* considera soltanto le coppie con supporto gold positivo, mantenendo nel conteggio degli errori anche le annotazioni sconosciute al modello.

Si riporta inoltre l'accuracy congiunta, definita come la frazione di parole per le quali risultano corretti sia UPOS sia l'intero insieme FEATS, affiancando alle misure complessive le metriche dei singoli attributi per esaminarne le differenze di comportamento.

2.7.3 Aggregazione e analisi degli errori

Per ogni configurazione si riportano media e deviazione standard campionaria sui tre seed [14], conservando le misure delle singole prove per rendere verificabili gli scarti osservati. Le variazioni UPOS tra le due fasi vengono calcolate appaiando i risultati per architettura e seed ed espresse in punti percentuali, mentre l'analisi degli errori esamina le principali confusioni UPOS e le coppie FEATS con più falsi positivi e falsi negativi, affiancate dalle metriche per classe.

2.8 Confronto con UDPipe 2

Il riferimento esterno è il modello UDPipe 2 `italian-isdt-ud-2.15-241121`, valutato sulle forme e sui confini gold del medesimo test, con le annotazioni grammaticali rimosse dall'input [12]. Le predizioni salvate vengono allineate parola per parola, verificando identità del modello e del test, e valutate con le stesse metriche UPOS adottate per BERT, così da mantenere omogenee le unità linguistiche e le convenzioni di calcolo.

Il punteggio UPOS F1 ufficiale con tokenizzazione gold è 98,75% [12] e coincide numericamente con l'accuracy quando l'allineamento delle parole è completo, mentre la macro-F1 viene ricalcolata dalle predizioni individuali. Il confronto esterno riguarda UPOS, mentre le FEATS vengono confrontate tra le architetture BERT dopo la seconda fase; poiché UDPipe 2 utilizza anche rappresentazioni contestuali da BERT multilingue, il riferimento riguarda sistemi completi nelle medesime condizioni di input [12].

2.9 Riproducibilità

Modello e tokenizer condividono la revisione registrata nei metadati, insieme agli hash dei dati, agli inventari, agli iperparametri e alle versioni dei pacchetti. Ogni prova conserva modello selezionato, cronologia di training, predizioni e metriche, includendo nella seconda fase percorso e hash dei pesi UPOS di origine; il riuso richiede configurazioni e pesi di partenza corrispondenti.

Bibliografia

[1] Christopher D. Manning. Part-of-speech tagging from 97% to 100%: Is it time for some linguistics? In *Computational Linguistics and Intelligent Text Processing*, volume 6608 of *Lecture Notes in Computer Science*, pages 171–189. Springer, 2011. doi:10.1007/978-3-642-19400-9_14.

[2] Universal Dependencies contributors. Universal POS tags. Documentazione di Universal Dependencies v2. URL: https://universaldependencies.org/u/pos/.

[3] Universal Dependencies contributors. CoNLL-U format. Specifiche di Universal Dependencies v2. URL: https://universaldependencies.org/format.html.

[4] Universal Dependencies contributors. UD Italian-ISDT. Treebank, release r2.15, 2024. URL: https://github.com/UniversalDependencies/UD_Italian-ISDT/tree/r2.15.

[5] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi:10.1109/TKDE.2008.239.

[6] Barbara Plank, Anders Søgaard, and Yoav Goldberg. Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. In *Proceedings of ACL 2016 (Volume 2: Short Papers)*, pages 412–418. Association for Computational Linguistics, 2016. doi:10.18653/v1/P16-2067.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186. Association for Computational Linguistics, 2019. doi:10.18653/v1/N19-1423.

[8] Yoav Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 57:345–420, 2016. doi:10.1613/jair.4992.

[9] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL: https://papers.nips.cc/paper_files/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[11] DBMDZ. BERT base italian cased. Model card, Hugging Face. Consultato il 7 novembre 2026. URL: https://huggingface.co/dbmdz/bert-base-italian-cased.

[12] ÚFAL, Charles University. UDPipe 2 models: Universal Dependencies 2.15. Documentazione dei modelli. Consultato il 7 novembre 2026. URL: https://ufal.mff.cuni.cz/udpipe/2/models.

[13] Dan Kondratyuk and Milan Straka. 75 languages, 1 model: Parsing Universal Dependencies universally. In *Proceedings of EMNLP-IJCNLP 2019*, pages 2779–2795. Association for Computational Linguistics, 2019. doi:10.18653/v1/D19-1279.

[14] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of EMNLP 2017*, pages 338–348. Association for Computational Linguistics, 2017. doi:10.18653/v1/D17-1035.

[15] Hugging Face. Token classification. Documentazione di Transformers, versione 4.46.3. URL: https://huggingface.co/docs/transformers/v4.46.3/tasks/token_classification.

[16] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of ICML 2001*, pages 282–289. Morgan Kaufmann, 2001. URL: https://repository.upenn.edu/cis_papers/159.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL: https://arxiv.org/abs/1711.05101.

[18] scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions. Documentazione di scikit-learn, versione 1.8. Consultato l'8 novembre 2026. URL: https://scikit-learn.org/1.8/modules/model_evaluation.html.